MODULE 29 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Securing Analytic Data in BigQuery

Authorised views, datasets and routines; row access policies; column-level security with policy tags; dynamic data masking; IAM conditions; and where Sensitive Data Protection fits.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 29 of 45

admin@sthithapragna.com

Share the answer, not the table. Then choose which rows and columns each person sees.

Six controls, six questions

CONTROL	THE QUESTION IT ANSWERS
IAM on datasets and tables	Who can open this at all?
Authorised views, datasets, routines	Can they see results without the source?
Row access policies	Which rows may this group see?
Policy tags	Which columns may they read?
Data masking	What does a protected column show them?
IAM conditions	Only until when, or only on which resources?

THE PATTERN

Most questions name one of these six questions. Find it, and the feature follows.

Share a view, not its source

- The view lives in a **separate dataset**, and is authorised to read the source dataset.
- Users need Data Viewer on the **view's dataset**, and Job User on a project.
- Source and view datasets must be in the **same location**.
- Up to **2,500** authorised resources per dataset. Materialized views can be authorised too.

> THE TRAP
>
> Granting users access to the source dataset as well defeats the whole point.

One view at a time, or a whole dataset?

AUTHORISED VIEW

One by one

- Authorise each view separately
- Fine for a handful of views
- Each new view needs its own grant

AUTHORISED DATASET

All views in it

- Every current and future view is authorised
- Recommended for new and multi-tenant designs
- Creating views there needs extra rights

THE TIEBREAKER

Many views, with more on the way: an authorised dataset. A single shared view: an authorised view.

Functions that read on your behalf

- Table functions, UDFs and stored procedures can be **authorised** on a dataset.
- Callers need permission to **run** the routine, not to read its tables.
- A **table function** with a parameter returns just the slice asked for.
- Authorised **stored procedures** get DDL and DML access. Trust callers fully.

> HONEST LIMIT
>
> CREATE OR REPLACE on an authorised routine removes its authorisation. Authorise it again.

One account's bills, for the app

```
1 CREATE OR REPLACE TABLE FUNCTION app_api.account_bills(acct STRING)
2 AS
3 SELECT bill_id, period_start, amount_gbp
4 FROM curated.bills
5 WHERE account_id = acct;
6
7 -- Then authorise app_api.account_bills on the curated dataset,
8 -- and grant the app's service account access to app_api only.
```

1 The function lives in its own dataset, app_api.

7-8 The app can call it, but cannot query curated.bills directly.

FOR LARKSPUR

The function returns three columns. Nothing else about the account ever leaves.

Row access policies

- **CREATE ROW ACCESS POLICY** names a table, who it grants, and a filter.
- **SESSION_USER()** in the filter shows each person only their own rows.
- Once a table has a policy, anyone in **no** grantee list sees no rows.
- Previews, BI Engine acceleration and partition pruning by the filter do not apply.

> THE TRAP
>
> Users who should still see everything need their own policy, with a TRUE filter.

Protecting vulnerable customers

```
1  CREATE ROW ACCESS POLICY all_but_psr ON curated.customers
2  GRANT TO ('group:billing-analysts@larkspur.energy')
3  FILTER USING (psr_flag = FALSE);
4
5  CREATE ROW ACCESS POLICY psr_team ON curated.customers
6  GRANT TO ('group:psr-team@larkspur.energy')
7  FILTER USING (TRUE);
```

3 Billing analysts never see customers on the priority services register.

7 The PSR team sees every row. Without this policy, they would see none.

REMEMBER

A person in both groups sees the union of the two filters, here every row.

Policy tags on sensitive columns

- A **taxonomy** holds policy tags, up to five levels deep. One tag per column.
- Turn on **enforcement**, then grant **Fine-Grained Reader** to those who may read.
- Without it, **SELECT *** fails. SELECT * EXCEPT the tagged columns still works.
- Newer options: **data governance tags** in Preview, and data policies set on a column.

> THE TRAP
>
> Taxonomies are regional. The taxonomy must sit in the same location as the table.

What a masked column can show

RULE	SHOWS	JOINABLE?
Nullify or default value	NULL, or 0 and empty strings	No
SHA-256 hash	The same hash every time	Yes, across queries
Random hash	A hash salted per query	Only within one query
Email mask	The domain, not the name	No
First four or last four	Four characters, then XXXXX	No
Date year mask or custom	The year only, or your own routine	Custom, if deterministic

THE TRAP

SHA-256 is deterministic, so it can be guessed from known values. Random hash is safer.

Fine-Grained Reader or Masked Reader?

FINE-GRAINED READER

Sees real values

- Granted on the policy tag
- For the few who truly need it
- Such as the fraud investigators' lead

MASKED READER

Sees masked values

- Granted on the data policy
- Existing queries keep working
- Neither role means no access at all

REMEMBER

When several masking rules apply, a fixed order picks one. A custom routine ranks highest, nullify lowest.

Access with conditions attached

- **request.time**: access that expires, such as an auditor until month end.
- **resource.name**: only datasets or tables with a given name or prefix.
- **Resource tags** on datasets, tables and views, such as environment equals prod.
- Not on authorised view, dataset or routine grants, and not for rows or columns.

> THE TRAP
>
> A loose prefix condition can grant far more than intended. Write conditions narrowly.

From sensitive data to safe sharing

01

Find

Sensitive Data Protection discovery profiles every table.

02

Classify

Policy tags on the sensitive columns.

03

Protect

Masking rules, row access policies.

04

Share

Authorised views, datasets and routines.

THE PATTERN

Discovery tells you where personal data is. The BigQuery controls decide who sees it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Results, but **no access to the source**	Authorised view or dataset
Many views, **more coming**	Authorised dataset
One account's data, **by parameter**	Authorised table function
Each group sees **its own rows**	Row access policy
Hide a column, but **still join on it**	SHA-256 masking rule
Access that **expires** on a date	IAM condition on request.time

THE PATTERN

Rows, columns, values, or time: name the dimension, then pick the feature.

> Westgrid's analysts need always-current regional demand totals computed from curated.readings_hh.
>
> They must never see meter-level rows, and must not be granted any access to the curated dataset.

What should the team set up?

A Data Viewer on the curated dataset, plus a row access policy

B A nightly copy of the regional totals into a separate table

C Data Viewer on the curated dataset, with an IAM condition on one table

D An authorised view that aggregates by region, in its own dataset

◆ PAUSE AND DECIDE

What should the team set up?

A Viewer plus row policy

Access to curated, and meter rows still visible.

B Nightly copy

Not always current, and one more pipeline.

C Viewer with a condition

Still grants access to curated, at meter level.

D Authorised view

Aggregates only, current, and no access to curated.

THE SIGNAL

Results without access to the source: an authorised view.

> Larkspur's fraud team must not see customers' email addresses. But they must compare them with hashed emails saved from earlier investigations.
>
> Their queries run on different days, against different tables.

Which masking rule should their data policy use?

A Nullify, so that the email column always comes back as NULL

B SHA-256 hash, so one email gives the same value every time

C Email mask, so that only the domain of each address is shown

D Random hash, so that each query uses its own fresh salt

◆ PAUSE AND DECIDE

Which masking rule should their data policy use?

A **Nullify**
Nothing left to compare.

B **SHA-256 hash**
Deterministic, so values match across queries and days.

C **Email mask**
Domains match across many different people.

D **Random hash**
Salt changes per query, so saved hashes never match.

THE SIGNAL

Hidden, but matchable across queries: a SHA-256 masking rule.

> The team added a row access policy on curated.customers so billing analysts see only rows where psr_flag is FALSE.
>
> Next morning, the data science team, who used to see every row, see no rows at all.

What should the team do?

A Grant the data science team Fine-Grained Reader on the table

B Drop the policy, and give billing analysts a filtered view instead

C Add a row access policy for data science, with a TRUE filter

D Turn on enforcement on the taxonomy that holds the policy tags

◆ PAUSE AND DECIDE

What should the team do?

A **Fine-Grained Reader**
A column role, needed only for subquery row policies.

B **Drop the policy**
Throws away the protection for billing analysts.

C **A TRUE policy**
Data science is granted every row again.

D **Enforce the taxonomy**
Column security, unrelated to rows.

THE SIGNAL

Users outside every row policy see nothing: grant them a TRUE filter.

> The customer app's service account must fetch one account's bills at a time, by account ID.
>
> It must never be able to query curated.bills directly, or read columns beyond bill ID, period and amount.

What should the team build?

A An authorised table function that takes the account ID

B Data Viewer on curated, with a row policy using SESSION_USER()

C An authorised view that selects every bill for every account

D A nightly export of each account's bills to Cloud Storage

◆ PAUSE AND DECIDE

What should the team build?

A Authorised table function

One account per call, three columns, no table access.

B Row policy on SESSION_USER

One service account is a single user, so it sees all.

C Authorised view of all

Can be read in bulk, all accounts at once.

D Nightly export

Stale, and millions of files to secure.

THE SIGNAL

Results by parameter, without source access: an authorised table function.

Module 29 in five lines

- **Authorised views**, datasets and routines share results, not sources.
- **Row access policies** filter rows. Anyone outside every policy sees none.
- **Policy tags** restrict columns. Fine-Grained Reader sees real values.
- **Masking**: SHA-256 to join across queries, random hash to stay safer.
- **IAM conditions** for time, names and tags, never for rows or columns.

COMING UP

Next: module 30, preparing data for ML with BigQuery ML.